

Tarea. Implementación de concurrencia en módulos

Estudiantes:

Juan Pablo Correa Rave

Juan Sebastián Briñez Orjuela

Simón Madrid Tamayo

Tecnología en Desarrollo de Software, Institución Universitaria Pascua Bravo
sistemas operativos
Grupo 100
Yexid Montenegro
Medellín

INTRODUCCIÓN

Para la elaboración de este trabajo, se eligió la **Opción 1: Desarrollo e implementación de un modelo de simulación de tráfico (SIGET) mediante programación concurrente.**

En las ciudades, los sistemas de control de tráfico reciben información de muchos sensores al mismo tiempo. El objetivo de este proyecto es simular cómo se gestionan esos datos en tiempo real. En lugar de permitir que los sensores envíen información sin control y colapsen el sistema, se diseñó un programa que actúa como un turno organizado: los sensores registran los vehículos detectados en una lista de espera y la central de control los va procesando uno por uno de forma clara, rápida y sin errores.

OBJETIVOS:

- **Objetivo General:**
 - Crear un programa que simule el trabajo conjunto entre varios sensores de tráfico y una central de control, asegurando que se comuniquen de forma ordenada y sin errores.
- **Objetivos Específicos:**
 - Organizar los datos en una lista de espera donde el primer reporte en llegar sea el primero en atenderse.
 - Usar un mecanismo de semáforo interno para avisar cuando hay espacio disponible para guardar datos y cuándo hay datos listos para procesar.

EXPLICACIÓN DEL CÓDIGO:

Estructura del Semáforo y variables de control.

```

1  #include <iostream>
2  #include <thread>
3  #include <queue>
4  #include <mutex>
5  #include <condition_variable>
6
7  using namespace std;
8  queue<int> buffer;
9  mutex mutexBuffer;
10
11 struct Semaforo {
12     mutex mtx;
13     condition_variable cv;
14     int cont;
15     Semaforo(int inicio) {
16         cont = inicio;
17     }
18     void esperar() {
19         unique_lock<mutex> lock(mtx);
20         while (cont == 0) {
21             cv.wait(lock);
22         }
23         cont--;
24     }
25     void avisar() {
26         unique_lock<mutex> lock(mtx);
27         cont++;
28         cv.notify_one();
29     }
30 };
31
32 Semaforo espacios(5);
33 Semaforo datos(0);
    
```

En este primer bloque se declara la cola compartida buffer donde se almacenarán los reportes, junto con mutexBuffer para proteger el acceso a la memoria. A continuación, se define la estructura Semaforo, la cual utiliza variables de condición para coordinar el paso de los hilos: esperar() pausa el hilo si no hay recursos disponibles, y avisar() libera un permiso. Finalmente, se inicializan dos semáforos: espacios (con capacidad para 5 elementos) y datos (iniciado en 0).

Funciones del Sensor (Productor) y Consumidor:

```

34
35 void sensor(int numero, int carros) {
36     espacios.esperar();
37     cout << "Sensor " << numero << " detecto " << carros << " carros" << endl;
38     mutexBuffer.lock();
39     buffer.push(carros);
40     mutexBuffer.unlock();
41
42     datos.avisar();
43 }
44 void consumidor(){
45     for (int i = 0; i < 3; i++) {
46         datos.esperar();
47         mutexBuffer.lock();
48         int carros = buffer.front();
49         buffer.pop();
50         mutexBuffer.unlock();
51         cout << "consumidor proceso" << carros << "carros" << endl;
52         espacios.avisar();
53     }
54 }

```

Estas dos funciones contienen la lógica principal de la simulación. En sensor, cada cámara verifica si hay espacio libre (espacios.esperar()), asegura la cola con mutexBuffer.lock() para guardar la cantidad de vehículos y notifica con datos.avisar(). Por su parte, la función consumidor se ejecuta en un bucle de 3 iteraciones que espera reportes (datos.esperar()), los extrae de la cola de forma segura y notifica que se ha liberado un espacio con espacios.avisar().

Función Principal (main) y ejecución de hilos

```

53 }
54 }
55 int main() {
56     thread hilo4(consumidor);
57     thread hilo1(sensor, 1, 5);
58     thread hilo2(sensor, 2, 8);
59     thread hilo3(sensor, 3, 3);
60
61     hilo1.join();
62     hilo2.join();
63     hilo3.join();
64     hilo4.join();
65
66     return 0;
67 }

```

En el bloque main se crean y lanzan los hilos en paralelo. Se inicia primero el hilo del consumidor (hilo4) para que esté listo a procesar eventos, seguido por los tres hilos de los sensores (hilo1, hilo2, hilo3), a los cuales se les asigna el número de sensor y los carros detectados. El uso de .join() garantiza que el programa principal espere a que todas las tareas concurrentes terminen ordenadamente antes de cerrar.

RESULTADOS EN PANTALLA:

```

C:\Users\USER\Downloads\Im X + v
Sensor Sensor 2 detecto 8 carrosSensor 3 detecto 3 carros
1 detecto 5 carros
consumidor proceso3carros
consumidor proceso5carros
consumidor proceso8carros

-----
Process exited after 1.02 seconds with return value 0
Presione una tecla para continuar . . .
    
```

Al ejecutar el programa, se observa cómo los sensores registran sus conteos de vehículos e inmediatamente la función consumidor toma cada reporte guardado en la cola y los procesa en orden sin generar errores de lectura ni pérdida de información.